

Một phương pháp kết hợp các thuật toán

Trương Dục Thừa
Trường Trung học Dương Châu, tỉnh Giang Tô

2008

Nguồn gốc: A method for algorithm composition

I0I-China-Candidate-Team-Papers/2008/Day2/4-Zhang-Yucheng/algorithm-composition.pdf

Abstract

Bài viết trình bày một phương pháp kết hợp thuật toán. Phương pháp này nghĩa là đem nhiều thuật toán cho cùng một bài toán áp dụng lần lượt vào các phần bổ sung cho nhau của bài toán ấy.

Bài viết phân tích chi tiết hai bài toán thứ vị và dùng thành công phương pháp kết hợp thuật toán này để giải chúng. Trong bài toán thứ nhất, ta kết hợp một thuật toán $\mathcal{O}(N^2)$ và một thuật toán $\mathcal{O}(NR)$, áp dụng chúng vào hai phần truy vấn của bài toán, thu được thuật toán $\mathcal{O}(N\sqrt{R})$. Trong bài toán thứ hai, ta dùng hai thuật toán $\mathcal{O}(N^2)$ trên ba phần sau khi chia bài toán gốc, thu được thuật toán $\mathcal{O}(N^{1.5})$.

Cuối cùng, bài viết tổng kết phương pháp này. Mỗi thuật toán đều có thể có ưu và nhược điểm riêng. Nếu kết hợp chúng và áp dụng vào các phần khác nhau của bài toán, ta có thể gộp được ưu điểm của chúng, bù trừ điểm yếu cho nhau và thu được một thuật toán tổng thể tốt hơn. Tư tưởng này hết sức quan trọng.

Từ khóa: kết hợp thuật toán, phương pháp.

1 Bài toán thứ nhất

1.1 Mô tả bài toán

Duy trì một tập S , ban đầu rỗng. Có hai loại thao tác trên tập này:

1. B X : chèn một số nguyên X vào tập S ; bảo đảm hiện tại X chưa tồn tại trong tập.
2. A Y : hỏi trong tập S , số nào có phần dư nhỏ nhất khi chia cho Y . Nếu có nhiều số có cùng phần dư, có thể chọn tùy ý một số.

Cần xử lý lần lượt N thao tác và tính đáp án cho mọi truy vấn. Cho phép dùng thuật toán offline.¹

$$1 \leq N \leq 40000, \quad 1 \leq X, Y \leq 500000.$$

1.2 Phân tích sơ bộ

Bài này yêu cầu thiết kế thuật toán duy trì một tập S . Trước hết ta xét một số thuật toán dễ nghĩ tới.

¹Nguồn bài: The 2006 ACM Asia Programming Contest – Shanghai.

Thuật toán dễ nghĩ nhất là mô phỏng trực tiếp các thao tác được quy định trong đề; ta gọi đó là thuật toán 1.0. Mỗi khi gặp truy vấn A Y, ta duyệt mọi số trong tập S hiện tại và tìm số có phần dư nhỏ nhất khi chia cho Y. Độ phức tạp thời gian là

$$\mathcal{O}(\text{số thao tác chèn} \times \text{số thao tác hỏi}),$$

nên trong trường hợp xấu nhất hiển nhiên sẽ quá chậm. Nhưng khi số thao tác chèn rất ít hoặc số thao tác hỏi rất ít, thuật toán này chạy nhanh.

Một thuật toán tốt hơn một chút cũng rất dễ nghĩ ra, gọi là thuật toán 1.1. Gọi $p(y)$ là số x trong tập S hiện tại làm cho $x \bmod y$ nhỏ nhất; đây cũng chính là đáp án của truy vấn A y. Vì cho phép xử lý offline, ta có thể gom trước tập T gồm mọi giá trị Y khác nhau xuất hiện trong các truy vấn, rồi duy trì giá trị $p(y)$ với từng $y \in T$. Mỗi khi chèn một số, ta dùng thời gian $\mathcal{O}(|T|)$ để cập nhật lần lượt các giá trị ấy. Độ phức tạp thời gian của thuật toán là

$$\mathcal{O}(\text{số thao tác chèn} \times |T|).$$

Vì $|T|$ cũng ở cấp $\mathcal{O}(N)$, thuật toán này vẫn chưa giải quyết trọn vẹn bài toán. Thật ra, tập T ở đây có thể hiểu là tập các truy vấn mà ta muốn duy trì. Nếu chỉ duy trì một phần các giá trị Y xuất hiện trong truy vấn, thời gian duy trì sẽ giảm, nhưng một số truy vấn sẽ không có sẵn đáp án.

1.3 Nắm đặc trưng bài toán để có thuật toán khác: thuật toán 1.2

Để giải bài toán này, ta cần nắm đặc trưng của nó và suy nghĩ sâu hơn.

Khi gặp truy vấn A Y, ta cần tìm trong tập S số x làm cho $x \bmod Y$ nhỏ nhất. Viết

$$x = kY + r, \quad 0 \leq r < Y.$$

Khi đó

$$x \bmod Y = (kY + r) \bmod Y = r.$$

Nói cách khác, ta phải tìm trong tập S số $kY + r$ làm cho r nhỏ nhất.

Nếu cố định k , bài toán trở thành tìm phần tử nhỏ nhất của S trong khoảng $[kY, (k+1)Y)$. Vì vậy, ta dễ nghĩ đến thuật toán sau: duyệt k , tìm phần tử nhỏ nhất trong khoảng $[kY, (k+1)Y)$ tương ứng, rồi chọn đáp án tốt nhất trong các giá trị nhỏ nhất ấy. Hình 1.1 mô tả trực quan quá trình này.

$[0, Y)$	$[Y, 2Y)$	$[2Y, 3Y)$
giá trị nhỏ nhất là 2	giá trị nhỏ nhất là Y	giá trị nhỏ nhất là $2Y + 1$

Hình 1.1. Các số tự nhiên được chia thành những khoảng độ dài Y. Trong mỗi khoảng $[kY, (k+1)Y)$, ta lấy phần tử nhỏ nhất thuộc S, rồi chọn giá trị tối ưu trong các phần tử ấy. Ở ví dụ này, giá trị tối ưu là Y, vì $Y \bmod Y = 0$.

Gọi R là giá trị lớn nhất có thể được chèn. Theo đề bài, $R = 500000$. Dễ thấy có $\mathcal{O}(R/Y)$ giá trị k khác nhau.

Bằng cách này, ta đã chuyển bài toán tìm số có phần dư nhỏ nhất khi chia cho Y thành nhiều bài toán tìm số nhỏ nhất trong một khoảng cho trước.

Bài toán sau rất quen thuộc và có thể giải bằng cây đoạn.² Xây dựng một cây đoạn trên $[0, R]$. Tại mỗi nút $[l, r]$ của cây, lưu số nhỏ nhất trong $S \cap [l, r]$. Vì ta biết trước mọi số sẽ được chèn, ta có thể rời rạc hóa $[0, R]$ và chỉ giữ lại $\mathcal{O}(N)$ điểm sẽ được chèn; khi đó mỗi thao tác chỉ cần thời gian $\mathcal{O}(\lg N)$.

Tuy nhiên, trong cùng mức thời gian, cây đoạn hỗ trợ nhiều thao tác hơn. Ở đây ta chỉ dùng thao tác hỏi giá trị nhỏ nhất trong một đoạn và chèn một số; hơn nữa, khi chèn tại vị trí pos, số được chèn cũng

²Về cây đoạn, có thể tham khảo *Introduction to Algorithms* và các bài viết liên quan của đội tuyển tập huấn quốc gia.

chính là pos. Vì vậy dùng cây đoạn có vẻ hơi “lãng phí”. Có thể ta tìm được một thuật toán hỗ trợ ít thao tác hơn nhưng hiệu quả cao hơn.

Hỏi số nhỏ nhất trong tập S thuộc khoảng $[a, b]$ có thể xem là hỏi số nhỏ nhất không nhỏ hơn a , ký hiệu là $q(a)$. Nếu không có số nào không nhỏ hơn a , ta đặt $q(a) = +\infty$. Rõ ràng, nếu $q(a) > b$ thì trong khoảng $[a, b]$ không có số nào thuộc S ; ngược lại, $q(a)$ chính là số nhỏ nhất trong $[a, b]$. Hình 1.2 cho một ví dụ cụ thể.

a	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$q(a)$	2	2	2	7	7	7	7	7	8	12	12	12	12	$+\infty$	$+\infty$	$+\infty$

Hình 1.2. Ở đây $S = \{2, 7, 8, 12\}$ và $Y = 5$; các giá trị $q(a)$ được ghi bên dưới từng vị trí.

Dễ quan sát rằng với nhiều giá trị a liên tiếp, $q(a)$ bằng nhau. Nếu S rỗng thì với mọi số tự nhiên a , $q(a) = +\infty$. Ngược lại, sắp xếp các số trong S thành

$$p_1 < p_2 < p_3 < \dots < p_n \quad (n \geq 1),$$

vì các số được chèn bảo đảm không lặp. Khi đó $q(a) = p_1$ với $a \in [0, p_1]$, $q(a) = p_2$ với $a \in [p_1 + 1, p_2]$, và cứ tiếp tục như vậy; sau giá trị cuối cùng, $q(a) = +\infty$.

Khi chèn một số X , giả sử X nằm trong khoảng $[s, t]$ mà trên đó $q(a) = t$. Như Hình 1.3, sau khi chèn, với $a \in [s, X]$ ta có $q(a) = X$, còn với $a \in [X + 1, t]$ ta có $q(a) = t$. Tức là khoảng $[s, t]$ bị tách thành hai khoảng $[s, X]$ và $[X + 1, t]$.

trước khi chèn X	$[s, t]$ có cùng giá trị đại diện t
sau khi chèn X	$[s, X]$ có đại diện X , còn $[X + 1, t]$ có đại diện t

Hình 1.3. Khi chèn số X , khoảng $[s, t]$ được tách thành $[s, X]$ và $[X + 1, t]$.

Vì chỉ có thao tác chèn, ta luôn tách các khoảng chứ không gộp chúng. Nếu cho thời gian chạy ngược và xử lý mọi thao tác theo thứ tự từ sau ra trước, các khoảng sẽ chỉ bị gộp lại. Khi trả lời truy vấn, ta chỉ cần tìm một số thuộc khoảng nào. Điều này gợi nhớ đến cấu trúc hợp nhất–tìm kiếm (disjoint-set union).³

Ta xem mỗi khoảng như một tập và duy trì cho mỗi tập giá trị q tương ứng của khoảng ấy. Thao tác gộp tập và tìm tập dùng thuật toán kinh điển, có thể đạt thời gian trung bình gần $\mathcal{O}(1)$ cho mỗi thao tác. Để thuận tiện, dưới đây ta xem xấp xỉ mỗi thao tác mất $\mathcal{O}(1)$.

Quay lại bài toán gốc. Mỗi thao tác chèn chỉ cần $\mathcal{O}(1)$ thời gian. Với một truy vấn A Y, cần hỏi $\mathcal{O}(R/Y)$ khoảng. Vì $Y \geq 1$, số khoảng này cũng chỉ bị chặn bởi $\mathcal{O}(R)$. Dù mỗi khoảng có thể xử lý trong $\mathcal{O}(1)$, thời gian cho một truy vấn vẫn lên tới $\mathcal{O}(R)$. Tổng độ phức tạp thời gian của thuật toán 1.2 là $\mathcal{O}(NR)$.

Hiển nhiên, giống thuật toán 1.0 và 1.1 đã nêu, thuật toán 1.2 cũng chưa giải được bài toán, thậm chí nhìn qua còn chậm hơn.

1.4 Dùng đồng thời thuật toán 1.1 và thuật toán 1.2

Hãy phân tích nút thắt của thuật toán 1.2. Nút thắt nằm ở chỗ khi xử lý truy vấn A Y, số khoảng cần xét có thể rất nhiều; điều này xảy ra khi Y khá nhỏ. Số giá trị k khác nhau là $\mathcal{O}(R/Y)$ và đã được quyết định ngay từ đầu, nên rất khó giảm số khoảng phải hỏi.

Nhưng ta chú ý rằng nút thắt chỉ xuất hiện khi Y nhỏ. Trong đa số trường hợp, số khoảng cần xử lý ít hơn. Khi $Y > \sqrt{R}$, ta có $R/Y < \sqrt{R}$. Khi đó thời gian cho N truy vấn là $\mathcal{O}(N\sqrt{R})$. Với quy mô của bài này, độ phức tạp ấy đã chấp nhận được.

³Về disjoint-set union, có thể tham khảo *Introduction to Algorithms* và các bài viết liên quan của đội tuyển tập huấn quốc gia.

Vì vậy, ta có thể chia truy vấn thành hai phần theo độ lớn của Y . Gọi K là ngưỡng phân chia. Khi $Y > K$, ta tiếp tục dùng thuật toán 1.2; khi $Y \leq K$, ta dùng thuật toán khác.

Thuật toán 1.2 có thể giải phần lớn truy vấn theo Y ; các giá trị Y còn lại sẽ ít. Nhớ lại thuật toán 1.1 ở trên: khi số giá trị Y cần duy trì nhỏ, nó rất tốt. Vì thế với $Y \leq K$, ta vừa vặn có thể dùng thuật toán 1.1. Lúc này đặt tập T của thuật toán 1.1 là $[1, K]$, tức là duy trì đáp án cho mọi truy vấn có $1 \leq Y \leq K$.

Từ đó thu được thuật toán 1.3:

1. Trước hết, xử lý các thao tác theo thứ tự xuôi để trả lời các truy vấn $Y \leq K$. Mỗi lần chèn, cập nhật $p(i)$ với mọi $1 \leq i \leq K$, mất $\mathcal{O}(K)$ thời gian. Mỗi truy vấn $Y \leq K$ hiển nhiên trả lời trong $\mathcal{O}(1)$.
2. Sau đó, xử lý các thao tác theo thứ tự ngược để trả lời các truy vấn $Y > K$. Mỗi thao tác chèn ở chiều xuôi trở thành một thao tác gộp hai tập ở chiều ngược, mất $\mathcal{O}(1)$. Khi hỏi A Y , ta tìm số nhỏ nhất trong $\mathcal{O}(R/Y)$ khoảng; với mỗi khoảng $[a, b]$, ta tìm tập chứa a , mất $\mathcal{O}(1)$. Vì $Y > K$, độ phức tạp của truy vấn là $\mathcal{O}(R/K)$.

Tổng độ phức tạp thời gian của thuật toán 1.3 là

$$\mathcal{O}\left(NK + \frac{NR}{K}\right),$$

trong đó K là ngưỡng do ta đặt. Xem N và R là hằng số trong phép tối ưu theo K , để thấy tổng thời gian nhỏ nhất khi $K = \sqrt{R}$, đạt

$$\mathcal{O}(N\sqrt{R}).$$

Trong bài này, N lớn nhất là 40000, $R = 500000$, nên $N\sqrt{R}$ lớn nhất xấp xỉ 28284271; thuật toán này hoàn toàn giải được bài toán.

1.5 Tiểu kết

Với bài này, sau suy nghĩ ban đầu, ta thu được hai thuật toán thô sơ: thuật toán 1.0 và thuật toán 1.1. Chúng có biểu hiện tốt trên một số dữ liệu, nhưng trường hợp xấu nhất đều quá chậm, không giải quyết trọn vẹn bài toán. Cần chú ý rằng thuật toán 1.1 chạy tốt khi $|T|$ nhỏ, tức là khi số truy vấn cần duy trì ít.

Sau đó, bằng cách nắm đặc trưng của bài toán từ mục tiêu làm phần dư khi chia cho một số là nhỏ nhất, ta thu được thuật toán 1.2. Thuật toán 1.2 nhanh khi giá trị Y trong truy vấn tương đối lớn, nhưng vẫn chưa giải quyết trọn vẹn bài toán.

Thuật toán 1.1 và thuật toán 1.2 nếu dùng riêng lẻ đều không đủ, nhưng ta nhận ra chúng có thể giải hai phần bổ sung cho nhau của bài toán. Theo độ lớn của Y , ta chia các truy vấn thành hai phần: dùng thuật toán 1.1 cho $Y \leq \sqrt{R}$ và dùng thuật toán 1.2 cho $Y > \sqrt{R}$. Cách làm này giải quyết hoàn toàn bài toán.

Có thể thấy trọng điểm của lời giải là không dùng một thuật toán thống nhất, mà đồng thời dùng hai thuật toán của cùng bài toán để giải hai phần bổ sung cho nhau.

2 Bài toán thứ hai

2.1 Mô tả bài toán

Cho N điểm trên một mặt phẳng. Hỏi có bao nhiêu hình chữ nhật có các cạnh song song với trục tọa độ và có bốn đỉnh là bốn điểm bất kỳ trong N điểm đã cho.⁴

$$1 \leq N \leq 250000.$$

⁴Nguồn bài: MIT Individual Contest 2007, SPOJ RECTANGL.

Giới hạn thời gian cho mỗi điểm kiểm thử tối đa là 30 giây.

2.2 Phân tích sơ bộ

Dù giới hạn thời gian của bài này rất dài, N lớn nhất vẫn là 250000. Để giải bài toán, ta kỳ vọng thiết kế được thuật toán có độ phức tạp thấp hơn $\mathcal{O}(N^2)$.

Vì hình chữ nhật cần có cạnh song song với trục tọa độ, điều kiện chủ yếu là các tọa độ bằng nhau; ta chỉ quan tâm đến quan hệ tương đối giữa các tọa độ. Do đó có thể rời rạc hóa tọa độ các điểm. Như Hình 2.1, sau khi rời rạc hóa, ta nhận được một lưới mà trường hợp xấu nhất có kích thước N^2 , còn các điểm đầu vào nằm trên các nút lưới.

Hình 2.1. Rời rạc hóa 12 điểm cho ra một lưới 4×5 .

Hiển nhiên, bốn điểm tạo thành một hình chữ nhật sẽ có hai điểm nằm trên một hàng của lưới và hai điểm nằm trên một hàng khác. Vì vậy, ta có thể tính số hình chữ nhật được tạo bởi điểm trên từng cặp hàng của lưới, rồi cộng các kết quả lại để được đáp án.

2.3 Một thuật toán dễ nghĩ tới

Một thuật toán $\mathcal{O}(N^2)$, gọi là thuật toán 2.1, không khó nghĩ ra. Ta lần lượt lấy từng cặp hàng i, j của lưới với $i < j$ làm biên trên và biên dưới của hình chữ nhật, rồi tính xem có thể tạo được bao nhiêu hình chữ nhật. Khi tính, trước hết duyệt một hàng i và đưa chỉ số cột của các phần tử trên hàng này vào bảng băm. Sau đó duyệt hàng còn lại j và đếm trong hàng j có bao nhiêu điểm có chỉ số cột nằm trong bảng băm. Cách làm này chính là tính số cặp điểm ở hai hàng có cùng chỉ số cột. Rõ ràng, nếu có s cặp điểm có cùng chỉ số cột, thì các điểm trên hai hàng này có thể tạo thành

$$\binom{s}{2}$$

hình chữ nhật. Cuối cùng cộng số hình chữ nhật trên mọi cặp hàng là được đáp án.

Hình 2.2. Khi con trỏ i ở hàng 1 và j ở hàng 3, hai hàng này có 3 cặp điểm cùng chỉ số cột. Vì vậy chúng tạo được $\binom{3}{2} = 3$ hình chữ nhật.

Với cách làm này, ta cần duyệt một hàng trong $\mathcal{O}(N)$ hàng; với mỗi hàng, phải xử lý $\mathcal{O}(N)$ điểm. Mỗi điểm trong $\mathcal{O}(N)$ điểm ấy nhiều nhất được đưa vào bảng băm một lần hoặc được kiểm tra xem tọa độ cột có nằm trong bảng băm không một lần. Vì vậy độ phức tạp thời gian là $\mathcal{O}(N^2)$. Thuật toán này chưa đạt kỳ vọng, nhưng khi số hàng của lưới ít, nó rất tốt.

2.4 Chia bài toán thành ba phần

Có thể nhận thấy rằng khi số điểm trên mỗi hàng của lưới đều nhiều, do tổng số điểm bị giới hạn, số hàng của lưới sẽ nhỏ. Vì vậy, ta phân loại các hàng theo số điểm trên hàng. Gọi K là ngưỡng phân chia. Nếu số điểm trên hàng x lớn hơn K , ta gọi hàng x là loại A; ngược lại là loại B. Khi đó bài toán hiển nhiên được chia thành ba phần:

1. Có bao nhiêu hình chữ nhật được tạo bởi các điểm trên hai hàng loại A?
2. Có bao nhiêu hình chữ nhật được tạo bởi các điểm trên hai hàng loại B?
3. Có bao nhiêu hình chữ nhật được tạo bởi một hàng loại A và một hàng loại B?

Dễ thấy số hàng loại A nhất định nhỏ hơn N/K . Chứng minh rất đơn giản. Giả sử số hàng loại A ít nhất là N/K . Vì mỗi hàng có hơn K điểm, tổng số điểm trên các hàng loại A sẽ lớn hơn

$$\frac{N}{K} \cdot K = N.$$

Nhưng trên thực tế số điểm trong các hàng loại A chắc chắn không vượt quá N , mâu thuẫn.

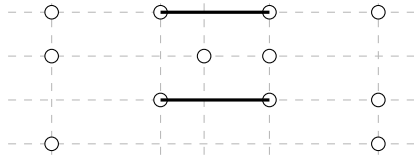
Với ràng buộc số hàng loại A khá ít, ta có thể dùng thuật toán 2.1 cho phần 1.

Chú ý trong thuật toán 2.1, chỉ cần duyệt trước một hàng; việc duyệt hàng còn lại về độ phức tạp tương đương với quét qua mọi điểm. Điều này cho phép khi xử lý phần 1, ta “tiện thể” xử lý luôn phần 3 mà không ảnh hưởng đến độ phức tạp.

Còn với phần 2, ta có một ràng buộc mới: số điểm trên mỗi hàng không vượt quá K , tức là mỗi hàng có ít điểm. Ta nắm đặc trưng này để thiết kế một thuật toán tốt hơn cho trường hợp số điểm trên mỗi hàng ít.

2.5 Thiết kế thuật toán khác cho phần 2: thuật toán 2.2

Trong phần 2, số điểm trên mỗi hàng ít cũng có nghĩa là số đoạn thẳng nhận hai điểm bất kỳ khác nhau trên cùng hàng làm hai đầu mút cũng ít. Nếu lấy một điểm làm đầu mút phải của đoạn l , vì một hàng có nhiều nhất K điểm, đầu mút trái của l có $\mathcal{O}(K)$ lựa chọn. Do đó, nếu lấy mọi điểm trong $\mathcal{O}(N)$ điểm làm đầu mút phải, sẽ có $\mathcal{O}(KN)$ đoạn thẳng.



Hình 2.3. Đoạn $[2, 4]$ xuất hiện 2 lần, tương ứng với hai đoạn được tô đậm.

Hiển nhiên, nếu gom các đoạn trên mọi hàng lại và với mỗi loại đoạn $[a, b]$ thống kê số lần xuất hiện s của nó, trong đó a và b là chỉ số cột của hai điểm trên cùng một hàng, thì đáp án là

$$\sum_{[a,b]} \binom{s_{a,b}}{2}.$$

Để đếm số lần xuất hiện của các đoạn như vậy, cách dễ nghĩ tới là dùng bảng băm, độ phức tạp thời gian $\mathcal{O}(KN)$. Nhưng cần chú ý rằng độ phức tạp không gian cũng là $\mathcal{O}(KN)$.

Một cách giảm độ phức tạp không gian không khó nghĩ ra là: trước hết cố định đầu mút phải b , rồi xét chung mọi đoạn có đầu mút phải là b và đưa các đầu mút trái của chúng vào bảng băm.

Vì lúc này chỉ cần băm một đầu mút, không gian giảm xuống còn $\mathcal{O}(N)$. Mỗi điểm vẫn chỉ được xét làm đầu mút phải một lần như trước, nên độ phức tạp thời gian không đổi, vẫn là $\mathcal{O}(KN)$.

2.6 Giải quyết bài toán

Đến đây cả ba phần đều đã có lời giải tốt. Ta hợp chúng lại để xét. Với phần 1 và phần 3, ta dùng thuật toán 2.1, độ phức tạp thời gian là

$$\mathcal{O}\left(\frac{N^2}{K}\right).$$

Với phần 2, ta dùng thuật toán 2.2, độ phức tạp thời gian là $\mathcal{O}(KN)$. Tổng độ phức tạp thời gian là

$$\mathcal{O}\left(\frac{N^2}{K} + KN\right).$$

Khi lấy $K = \sqrt{N}$, độ phức tạp đạt nhỏ nhất là

$$\mathcal{O}(N^{1.5}),$$

có thể giải được bài toán.

2.7 Tiểu kết

Sau phân tích sơ bộ đơn giản, ta rất dễ thu được thuật toán 2.1. Nó không thể giải trọn vẹn bài toán, nhưng khi số hàng ít thì rất tốt. Dựa vào ưu điểm này của thuật toán 2.1, ta chia bài toán thành ba phần theo số điểm trên mỗi hàng. Với phần 1 và phần 3, ta dùng thuật toán 2.1. Với phần 2, dựa trên đặc trưng của nó, ta thiết kế một thuật toán 2.2 rất nhanh cho riêng phần ấy. Sau đó ta dùng đồng thời thuật toán 2.1 và thuật toán 2.2 để thu được thuật toán có tổng độ phức tạp $\mathcal{O}(N^{1.5})$, qua đó giải được bài toán.

Nếu chỉ dùng riêng thuật toán 2.1 hoặc thuật toán 2.2, trong trường hợp xấu nhất ta đều nhận được thuật toán $\mathcal{O}(N^2)$, không thể giải trọn vẹn bài toán. Điều này cho thấy tầm quan trọng của phương pháp kết hợp thuật toán trong lời giải bài này.

Giống bài toán thứ nhất, bài này cũng kết hợp hai thuật toán tương đối đơn giản và áp dụng chúng vào các phần khác nhau của bài toán, nhưng cách chia phần không rõ ràng như trước. Để chia bài toán được như vậy, ta cần khả năng quan sát nhạy bén và nền tảng cơ bản vững chắc.

3 Tổng kết

Một bài toán thường có thể được xem là do nhiều phần hợp thành. Cần chú ý rằng các “phần” nói ở đây là những phần tương đối song song với nhau. Thông thường, ta dùng một thuật toán thống nhất cho các phần ấy. Nhưng đôi khi một bài toán có thể giải bằng nhiều thuật toán, và khi các thuật toán này được áp dụng vào những phần có đặc trưng khác nhau của bài toán, chúng đem lại hiệu quả khác nhau. Khi đó ta có thể kết hợp các thuật toán: với từng phần khác nhau của bài toán, căn cứ vào đặc trưng của phần ấy để chọn thuật toán tốt hơn cho riêng phần đó. Đây chính là phương pháp kết hợp thuật toán mà bài viết trình bày.

Với hai bài toán trong bài viết, ta đều dùng phương pháp này. Trong bài toán thứ nhất, ta thu được hai thuật toán có trường hợp xấu nhất lần lượt là $\mathcal{O}(N^2)$ và $\mathcal{O}(NR)$. Cả hai đều không tự giải được bài toán, nhưng chúng đều có hiệu quả tốt trên hai phần riêng của bài toán. Vì vậy ta dùng hai thuật toán ấy cho hai phần tương ứng và cuối cùng thu được thuật toán $\mathcal{O}(N\sqrt{R})$, qua đó giải bài toán khá tốt. Bài toán thứ hai cũng tương tự: ta kết hợp hai thuật toán có trường hợp xấu nhất $\mathcal{O}(N^2)$ để thu được một thuật toán $\mathcal{O}(N^{1.5})$.

Ta chú ý rằng sau khi gộp hai thuật toán lại, ta “thần kỳ” nhận được một thuật toán tốt hơn. Nguyên nhân là hai thuật toán này có ưu thế bổ sung cho nhau; khi chia bài toán thành nhiều phần và dùng thuật toán tốt hơn cho từng phần theo đặc trưng của phần đó, ưu điểm của hai thuật toán được kết hợp lại.

Mỗi thuật toán đều có ưu và nhược điểm riêng. Nếu ta bù trừ điểm yếu cho nhau và tận dụng đầy đủ ưu điểm của chúng, có thể sẽ thu được thuật toán tổng thể tốt hơn. Tư tưởng bù trừ ưu nhược này rất quan trọng.

Bài viết trình bày một lớp phương pháp kết hợp thuật toán. Với tư cách một phương pháp, ta có thể chọn dùng nó khi giải bài. Đồng thời, trong quá trình giải bài, việc không ngừng tổng kết để hình thành những phương pháp có tính tổng quát cũng rất quan trọng.

Tài liệu tham khảo

1. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, *Introduction to Algorithms*.
2. Các bài viết và bài tập của đội tuyển tập huấn quốc gia giai đoạn 2005–2007.